

the strict consistency guarantees of traditional relational databases.

Distributed database management systems (DDBMS): Modern distributed databases are built on distributed database management systems. DDBMSs provide a framework for managing data across multiple nodes, handling data partitioning, replication, consistency, and query processing in a distributed fashion. They often incorporate techniques like distributed transactions, consensus protocols, and replication strategies to ensure data integrity and fault tolerance.

Cloud databases: Cloud computing and storage technologies have revolutionised the way databases are deployed and managed. Cloud databases leverage the scalability, elasticity, and accessibility of cloud infrastructure to provide distributed database services. They offer features like automated data replication, load balancing, and flexible scalability, making it easier for organisations to handle large scale distributed data.

Blockchain and distributed ledgers: Blockchain technology introduced a new paradigm for distributed databases. Blockchain is a decentralised, append-only ledger that uses consensus algorithms to achieve data consistency across a network of nodes. It ensures trust and immutability by storing data in a distributed and tamper-proof manner. While initially associated with cryptocurrencies, blockchain concepts are being explored for various applications beyond finance.

These are some of the significant stages in the evolution of distributed databases. The field continues to evolve, with ongoing research and advancements in areas like distributed query optimisation, data replication strategies, fault tolerance, and distributed consensus algorithms.

Now that event driven architecture and microservices are getting popular, another key feature in the context of distributed databases and cloud

platforms is the relationship between a distributed and a polyglot database.

Computational components of a distributed database

Distributed databases consist of several computational components that work together to provide data storage, processing, and management across multiple nodes or servers. Here are the key computational components of a distributed database.

Data storage layer: The data storage layer is responsible for storing and managing the actual data within the distributed database. It includes the mechanisms and algorithms for data partitioning, replication, and distribution across multiple nodes. Each node in the distributed database typically stores a subset of the data, and the storage layer ensures that data is distributed optimally for scalability and fault tolerance.

Query processing and optimisation: The query processing and optimisation component handles the execution of queries or data manipulation operations on the distributed database. It receives user queries, breaks them down into subqueries, and optimises their execution across multiple nodes. The goal is to minimise data transfer between nodes and maximise query performance by leveraging parallel processing and data locality.

Distributed transaction manager: The distributed transaction manager ensures that transactions, which consist of multiple operations that must be executed atomically, consistently, and reliably, are properly managed in a distributed environment. It coordinates the execution of distributed transactions across multiple nodes, handles concurrency control to prevent conflicts, and ensures the durability and consistency of the data.

Distributed query coordinator: The distributed query coordinator acts as the central point of control for processing distributed queries. It receives and coordinates queries from clients, breaks them down into subqueries

that can be executed on individual nodes, and gathers and merges the results from various nodes to form the result set. The query coordinator also handles query optimisation and data distribution decisions.

Consensus and coordination protocols: In a distributed database, consensus and coordination protocols play a crucial role in achieving consistency and fault tolerance. These protocols enable nodes to agree on the state of the system, coordinate data replication, handle failures, and ensure that data modifications are properly replicated across nodes. Examples of consensus protocols include Paxos, Raft, and Distributed Two-Phase Commit.

Distributed lock manager: The distributed lock manager is responsible for managing locks on data items to ensure proper concurrency control in a distributed environment. It coordinates lock acquisition and release across multiple nodes to prevent conflicts and maintain data integrity. The lock manager also handles deadlock detection and resolution to avoid resource contention issues.

Data replication and synchronisation: Data replication is an important aspect of distributed databases for fault tolerance and data availability. The data replication and synchronisation component ensures that data updates are propagated to replicated copies across different nodes, maintaining consistency and durability. It handles mechanisms like replication protocols, conflict resolution, and data synchronisation algorithms.

Distributed monitoring and management: Distributed databases require monitoring and management tools to oversee the health, performance, and configuration of the distributed system. These tools collect metrics, monitor node status, detect and resolve failures, and provide insights into system behaviour. They may include distributed monitoring agents, dashboards, and alerting systems.